Name: Soham Chavan

PRN: 124B1F022

Class: IT A

Assignment No. 2

Aim:

To implement Quick Sort to efficiently sort movies on the StreamFlix platform based on user-selected parameters such as IMDB rating, release year, and watch time, while maintaining fast response times for large datasets.

Code:

```
import java.util.Random;

class Movie {
    String title;
    double imdbRating;
    int releaseYear;
    int watchTimeMinutes;

    Movie(String title, double imdbRating, int releaseYear, int watchTimeMinutes) {
        this.title = title;
        this.imdbRating = imdbRating;
        this.releaseYear = releaseYear;
        this.watchTimeMinutes = watchTimeMinutes;
    }

    public String toString() {
        return title + " | Rating:" + imdbRating + " | Year:" + releaseYear + " | WatchTime:"
+ watchTimeMinutes;
    }
}

public class QuickSortMovies {

    // key = 1 -> IMDB rating, key = 2 -> release year, key = 3 -> watch time
    public static void quickSort(Movie[] movies, int low, int high, int key) {
        if (low < high) {
            int pi = partition(movies, low, high, key);
            quickSort(movies, low, pi - 1, key);
            quickSort(movies, pi + 1, high, key);
        }
    }

    public static int partition(Movie[] movies, int low, int high, int key) {
        Movie pivot = movies[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
```

```
            if (compare(movies[j], pivot, key) > 0) {
                i++;
                swap(movies, i, j);
            }
        }
        swap(movies, i + 1, high);
        return i + 1;
    }

    public static int compare(Movie a, Movie b, int key) {
        switch (key) {
            case 1: return Double.compare(a.imdbRating, b.imdbRating);
            case 2: return Integer.compare(a.releaseYear, b.releaseYear);
            case 3: return Integer.compare(a.watchTimeMinutes, b.watchTimeMinutes);
            default: return 0;
        }
    }

    public static void swap(Movie[] movies, int i, int j) {
        Movie temp = movies[i];
        movies[i] = movies[j];
        movies[j] = temp;
    }

    public static void main(String[] args) {
        Movie[] movies = {
            new Movie("Inception", 8.8, 2010, 148),
            new Movie("Interstellar", 8.6, 2014, 169),
            new Movie("The Dark Knight", 9.0, 2008, 152),
            new Movie("Tenet", 7.5, 2020, 150),
            new Movie("Dunkirk", 7.8, 2017, 106),
            new Movie("Oppenheimer", 8.4, 2023, 180),
            new Movie("Parasite", 8.6, 2019, 132)
        };

        System.out.println("Original Movie List:");
        for (Movie m : movies) System.out.println(m);

        Movie[] byRating = movies.clone();
        quickSort(byRating, 0, byRating.length - 1, 1);
        System.out.println("\nSorted by IMDB Rating (Descending):");
        for (Movie m : byRating) System.out.println(m);

        Movie[] byYear = movies.clone();
        quickSort(byYear, 0, byYear.length - 1, 2);
        System.out.println("\nSorted by Release Year (Descending):");
        for (Movie m : byYear) System.out.println(m);

        Movie[] byWatchTime = movies.clone();
        quickSort(byWatchTime, 0, byWatchTime.length - 1, 3);
        System.out.println("\nSorted by Watch Time (Descending):");
        for (Movie m : byWatchTime) System.out.println(m);

        int largeSize = 100000;
        Movie[] largeMovies = new Movie[largeSize];
        Random rand = new Random(7);
        for (int i = 0; i < largeSize; i++) {
            largeMovies[i] = new Movie("Movie" + i, 1 + rand.nextDouble() * 9, 1970 +
rand.nextInt(55), 60 + rand.nextInt(120));
        }
```

```
        long start = System.nanoTime();
        quickSort(largeMovies, 0, largeMovies.length - 1, 1);
        long end = System.nanoTime();

        System.out.println("\nSorted " + largeSize + " movies by rating in " + (end - start) /
1000000.0 + " ms");
    }
}
```

Output:

```
Original Movie List:
Inception | Rating:8.8 | Year:2010 | WatchTime:148
Interstellar | Rating:8.6 | Year:2014 | WatchTime:169
The Dark Knight | Rating:9.0 | Year:2008 | WatchTime:152
Tenet | Rating:7.5 | Year:2020 | WatchTime:150
Dunkirk | Rating:7.8 | Year:2017 | WatchTime:106
Oppenheimer | Rating:8.4 | Year:2023 | WatchTime:180
Parasite | Rating:8.6 | Year:2019 | WatchTime:132

Sorted by IMDB Rating (Descending):
The Dark Knight | Rating:9.0 | Year:2008 | WatchTime:152
Inception | Rating:8.8 | Year:2010 | WatchTime:148
Parasite | Rating:8.6 | Year:2019 | WatchTime:132
Interstellar | Rating:8.6 | Year:2014 | WatchTime:169
Oppenheimer | Rating:8.4 | Year:2023 | WatchTime:180
Dunkirk | Rating:7.8 | Year:2017 | WatchTime:106
Tenet | Rating:7.5 | Year:2020 | WatchTime:150

Sorted by Release Year (Descending):
Oppenheimer | Rating:8.4 | Year:2023 | WatchTime:180
Tenet | Rating:7.5 | Year:2020 | WatchTime:150
Parasite | Rating:8.6 | Year:2019 | WatchTime:132
Dunkirk | Rating:7.8 | Year:2017 | WatchTime:106
Interstellar | Rating:8.6 | Year:2014 | WatchTime:169
Inception | Rating:8.8 | Year:2010 | WatchTime:148
The Dark Knight | Rating:9.0 | Year:2008 | WatchTime:152

Sorted by Watch Time (Descending):
Oppenheimer | Rating:8.4 | Year:2023 | WatchTime:180
Interstellar | Rating:8.6 | Year:2014 | WatchTime:169
The Dark Knight | Rating:9.0 | Year:2008 | WatchTime:152
Tenet | Rating:7.5 | Year:2020 | WatchTime:150
Inception | Rating:8.8 | Year:2010 | WatchTime:148
Parasite | Rating:8.6 | Year:2019 | WatchTime:132
Dunkirk | Rating:7.8 | Year:2017 | WatchTime:106

Sorted 100000 movies by rating in 41.777197 ms
```

Conclusion:

Quick Sort was implemented to sort movies based on multiple parameters using a generic comparison key. It uses the divide-and-conquer strategy with in-place partitioning around a pivot, giving it an average time complexity of O(n log n) and very good practical performance due to low constant factors and cache efficiency. This makes it suitable for a recommendation system that must sort large movie datasets quickly during peak traffic.